OOP C++ Programming: Finite State Machine Implementation for Robot Vacuum Cleaner

Valeriya Kostyukova
Nazarbayev University, SEDS, Robotics

I. Introduction

FINITE State Machines (FSMs) are widely used in robotics and embedded systems to model behavior using a set of defined states and transitions. To explore these principles in practice, I implemented an FSM-based behavior controller for a simulated robot vacuum cleaner.

The FSM is developed using object-oriented programming (OOP) in C++ so that each state, such as *Idle*, *Cleaning*, *Returning to Dock*, and *Charging*, are encapsulated within its own class. Transitions between states are triggered by events such as low battery levels, the completion of cleaning tasks, or user commands.

To visualize and interact with the FSM, a graphical user interface (GUI) is built using the Qt framework. The GUI includes a dynamic top panel that highlights the current FSM state, a battery indicator, and a map area where the robot navigates to clean randomly appearing dust particles. The simulation also includes a reactive cat character that responds to the robot's presence, providing an additional layer of environmental interaction (Fig. 1).

II. Task 2: C++ Based FSM Implementation

In this task, I implemented the robot vacuum cleaner logic using OOP approach in C++, following *WestWorld* FSM design pattern. This implementation allowed for encapsulated state behavior and clear transitions between different robot states.

FSM Structure and Design

The FSM was implemented using an abstract `State` base class that defines a common interface for all concrete states. Each robot state (e.g., `IdleState`, `CleaningState`, `ReturningState`, `ChargingState`) inherits from this base class and overrides its virtual methods such as `Enter()`, `Execute()`, and `Exit()`.

A central `StateMachine` class manages the current state and handles transitions. It holds a pointer to the current state object and invokes its methods based on internal logic or external triggers. State transitions are managed using dynamic memory allocation to create and delete state objects during runtime.

File Organization

The implementation was organized into modular C++ files, each playing a specific role in the FSM logic:

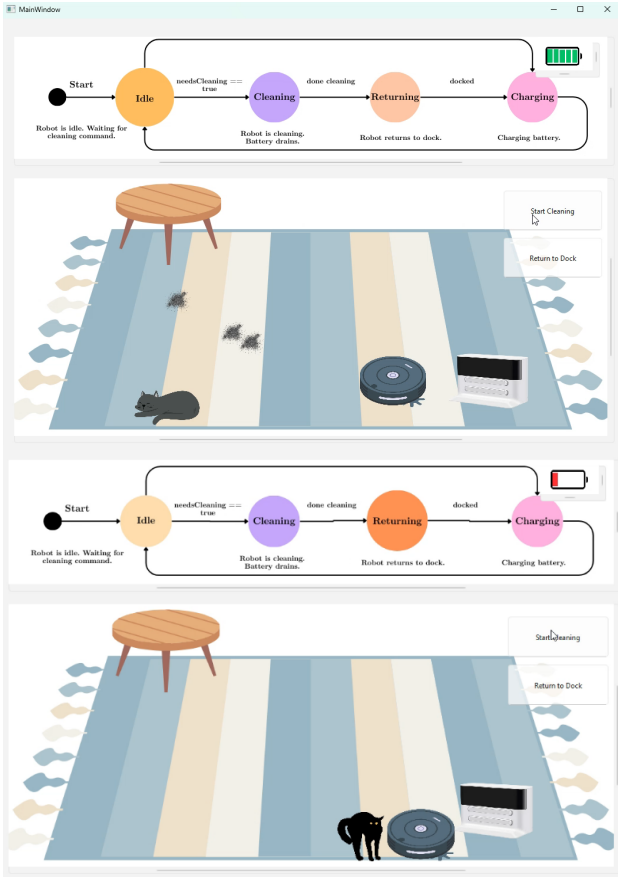

Fig. 1: GUI of the FSM-controlled robot vacuum cleaner.

- **`State.h`**: Declares the abstract base class `State<T>` using templates to allow the FSM to be used with any owner type. This class defines the virtual methods `Enter()`, `Execute()`, and `Exit()`, which are overridden by specific states.
- **`IdleState.h / IdleState.cpp`**: Implements the robot's idle behavior. In `Enter()`, the robot waits for the user to start cleaning. In `Execute()`, it checks for cleaning requests. If such a request is received, a transition to `CleaningState` is triggered. `Exit()` is used to perform any cleanup before moving to the next state.
- **`CleaningState.h / CleaningState.cpp`**: Handles the logic for virtually navigating to dust spots and cleaning them. This class updates the battery level during cleaning, and once all dust is

removed or the battery is low, it initiates a transition to `ReturningState`.

- **`ReturningToBaseState.h / ReturningToBaseState.cpp`**: This file contains the logic for moving the robot back to its charging dock. Upon reaching the dock, it signals the FSM to switch to `ChargingState`.
- **`ChargingState.h / ChargingState.cpp`**: Simulates the battery charging process. It updates the battery level and when fully charged transitions the robot back to `IdleState`.
- **`StateMachine.h`**: Implements the main FSM logic. It stores pointers to the current state and the previous state.
- **`RobotVacuum.h / RobotVacuum.cpp`**: Defines the robot agent class. It has an instance of `StateMachine<Robot>`, maintains the battery level and flags such as `needsCleaning`.
- **`main.cpp`**: Initializes the robot and runs the simulation loop. In this non-GUI version, it simulates time steps, monitors battery status, and feeds environmental triggers to the robot.

Key Techniques and Implementation Details

- **Polymorphism**: Used to define a unified interface across all state classes and ensure flexible transitions at runtime.
- **Dynamic memory management**: States were created dynamically and managed through pointers.
- **Encapsulation**: Each state encapsulates its own logic.
- **Debug printing**: For development and debugging, each state outputs entry, execution, and exit phases to standard output.
- **Singleton Pattern**: Each state class is implemented as a singleton, ensuring a single shared instance is reused throughout execution. This approach avoids unnecessary memory allocation and simplifies state transitions using static `Instance()` methods.

UML Diagram

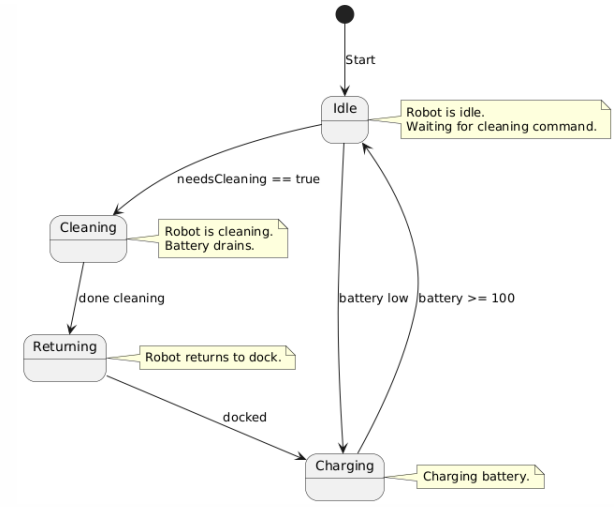

Fig. 2: PlantUML diagram of the robot vacuum cleaner FSM

To visualize the structure and behavior of the FSM, I created a UML state diagram using the PlantUML online tool. This diagram shows the representation of the robot's possible states and the events that trigger transitions between them (Fig. 2).

The main FSM states include:

- **Idle**: The default state, where the robot awaits cleaning commands.
- **Cleaning**: Activated when a cleaning request is received; the robot searches for dust and consumes battery.
- **Returning**: Initiated once the cleaning is done or the battery is low; the robot returns to its dock.
- **Charging**: The robot stays in this state until the battery reaches full charge (100%).

Each transition in the diagram is annotated with a specific condition or event, such as `needsCleaning == true`, `battery low`, or `done cleaning`.

III. Building the FSM with CMake and Running the Program

To organize the project and build process, I used `CMake`, a cross-platform build system. The configuration is specified in the `CMakeLists.txt` file, which defines the project settings, source files, and compilation options.

CMake Configuration

The following components were defined in the CMake configuration:

- **Project name**: `RobotVacuumFSM`
- **C++ Standard**: The code is compiled using the `C++17` standard.
- **Source files**: All FSM-related files such as `IdleState.cpp`, `CleaningState.cpp`, `ChargingState.cpp`, `ReturningToBaseState.cpp`, and the controller class `RobotVacuum.cpp` are included.
- **Executable**: The main application is compiled into an executable named `RobotVacuumFSM`.
- **Include directories**: The current directory is included to allow header file access.

Compiling the Program

The following section describes the process of compilig the project.

```
# 1. Navigating to the project directory
cd path/to/your/project

# 2. Creating and entering a build directory
mkdir build
cd build

# 3. Configuring the project using CMake
cmake ..

# 4. Building the project
cmake --build .
```

Execution Behavior

Running the compiled executable (`./RobotVacuumFSM.exe`) launches a text-based simulation of the robot vacuum cleaner. As it transitions between states, the program prints debug logs for:

- Entering a state
- Performing actions within a state (e.g., cleaning, charging)
- Exiting a state

These messages help visualize and verify the FSM's behavior in a terminal window, following an event-driven sequence.

As shown in Figure 3, the terminal output displays the robot's FSM behavior cycle by cycle. Initially, the robot starts in the `Idle` state. When dirt is detected (Cycle 1 and 10), it transitions to `Cleaning`, followed by `Returning` and `Charging`. Each state logs entry, actions (e.g., vacuuming, docking), and exit, along with battery level updates.

```
danissa@nlt061635:~/Desktop/CSCI502_Assignment4_VKostyukova/build$ ./RobotVacuum
FSM
[Idle] Entering idle mode...

=== Cycle 1 ===
[Main] Battery level: 100
[Idle] Time to clean!
[Idle] Leaving idle mode...
[Cleaning] Starting to clean...

=== Cycle 2 ===
[Main] Battery level: 100
[Cleaning] Vacuuming...
[Cleaning] Battery after cleaning: 90
[Cleaning] Done cleaning. Returning to dock.
[Cleaning] Finished cleaning.
[Returning] Heading back to base...

=== Cycle 3 ===
[Main] Battery level: 90
[Returning] Navigating to docking station...
[Returning] Docked.
[Returning] Leaving return-to-base state.
[Charging] Starting to charge...

=== Cycle 4 ===
[Main] Battery level: 90
[Charging] Charging battery...
[Charging] Fully charged. Switching to idle.
[Charging] Leaving charging state.
[Idle] Entering idle mode...

=== Cycle 5 ===
[Main] Battery level: 100

=== Cycle 6 ===
[Main] Battery level: 100

=== Cycle 7 ===
[Main] Battery level: 100

=== Cycle 8 ===
[Main] Battery level: 100

=== Cycle 9 ===
[Main] Battery level: 100

=== Cycle 10 ===
[Main] Dirt detected! Scheduling cleaning...
[Main] Battery level: 100
[Idle] Time to clean!
[Idle] Leaving idle mode...
[Cleaning] Starting to clean...
```

Fig. 3: Terminal output showing FSM state transitions and actions

IV. Qt Based FSM Implementation

To provide a more interactive and visual understanding of the FSM, the same robot vacuum logic was implemented using Qt. The GUI mimics an environment where a robot vacuum cleaner performs its cleaning task based on user input and system state.

Overview of the GUI

The application window consists of two main sections:

- **FSM Visualization (Top)**: A dynamic diagram showing the current FSM state of the robot. The current state node vibrates to indicate activity, providing real-time feedback of transitions (`Idle`, `Cleaning`, `Returning`, and `Charging`). The diagram resembles the one derived from the PlantUML tool (Fig. 4).
- **Environment Simulation (Bottom)**: A 2D environment with a carpet, dock station, robot vacuum, dust spots, and a sleeping cat. The robot moves toward dust points and reacts to user interactions, while the cat responds by moving away when the robot approaches (Fig. 5).

FSM Logic Integration

Although the FSM behavior is visually driven in the GUI, the logic follows the same structure as in the C++ OOP implementation. The robot transitions between FSM states based on conditions such as battery level and the completion of cleaning tasks. Each state is represented visually through a set of custom images that are animated using QTimer-based vibration.

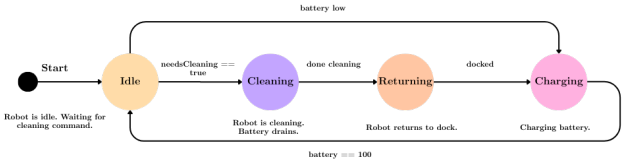

Fig. 4: Colorful FSM diagram made for Qt GUI.

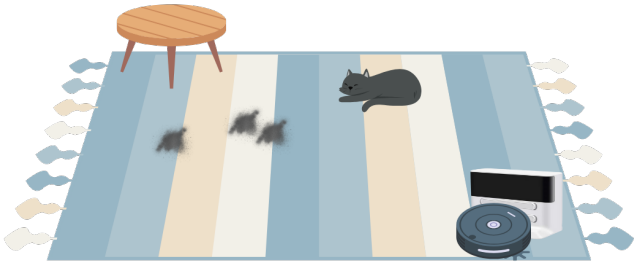

Fig. 5: Environment simulation: a carpet, dock station, robot vacuum, dust spots, and a sleeping cat.

Implementation Details

The entire Qt FSM-based GUI was implemented within the `MainWindow` class in `mainwindow.cpp`.

Initialization and Scene Setup: Upon launching the application, the constructor `MainWindow::MainWindow()` performs the following tasks:

- Sets up three `QGraphicsScenes`:
  - `fsmView` – displays the animated FSM diagram on the top.

  - batteryView – shows a dynamic battery level indicator.
  - mapView – displays the environment with robot, cat, dust, and furniture.
- Loads all initial pixmaps (robot, dock, cat, dust, carpet) and adds them to the appropriate scene.
- Initializes timers for FSM vibration, battery updates, idle cycle, and charging logic.
- Places a sleeping cat at a random position.

Robot and FSM Animation: The FSM states are animated using a list of image frames corresponding to each state. When a state becomes active, the FSM image is animated by cycling through its image list via a `QTimer`:

- `startFSMVibration()` starts the animation by loading a frame list.
- `updateFSMFrame()` updates the FSM image periodically every 200 ms.

```
void MainWindow::startFSMVibration
  (const QStringList &frames) {
  fsmFrames = frames;
  currentFSMFrame = 0;
  fsmTimer->start(200);
}

void MainWindow::updateFSMFrame() {
  ui->fsmView->scene()->clear();
  ui->fsmView->scene()->addPixmap
   (QPixmap(fsmFrames[currentFSMFrame]));
  currentFSMFrame=(currentFSMFrame+1)
   % fsmFrames.size();
}
```

The robot's movement is animated using another `QTimer` in `animateRobotTo()`, which interpolates the robot's position toward a goal using linear steps.

```
void MainWindow::animateRobotTo(QPointF
    position, std::function<void()>
    callback) {
     moveTimer->start(22); // ~22ms
     connect(moveTimer, &QTimer::timeout,
      this, [=]() {
     QPointF step = robotItem->pos() + 0.1
       * (position - robotItem->pos());
     robotItem->setPos(step);
     if (closeEnough(step, position)) {
            moveTimer->stop();
            callback();
        }
     });
}
```

Dust Cleaning Logic: When the `Start Cleaning` button is pressed, the function `cleanDustOneByOne()` is triggered. It recursively cleans dust spots one-by-one:

- The robot animates toward the next dust location.
- On arrival, the dust is hidden, and the battery is reduced.
- If the battery depletes or the cleaning is interrupted, the robot transitions to `Returning` state.

```
void MainWindow::on_startButton_clicked() {
    isInterrupted = false;
    cleanDustOneByOne(0);
}
```

Battery and Docking: The battery level is updated visually in real time. If it falls below a certain threshold or after cleaning, the robot returns to its docking station. The function `chargeBattery()` handles the step-by-step recharge process. After full charge, the FSM switches back to idle and dust appears again after 3 seconds.

```
void MainWindow::chargeBattery() {
    chargingStepTimer->start(1500);
    connect(chargingStepTimer,
     &QTimer::timeout, this, [=]() {
        batteryLevel += 20;
        updateBattery();
        if (batteryLevel >= 100) {
            chargingStepTimer->stop();
            updateFSMToIdle();
        }
    });
}
```

Battery images are switched dynamically:

```
batteryItem->setPixmap(QPixmap(levels[index]));
```

Cat Interaction Behavior: A sleeping cat is randomly placed on the carpet. When the robot approaches within 70 pixels:

- The cat transitions to an `alert pose` for 500 ms.
- Then, it switches to a `running image` and moves to a new location further from the robot.
- This logic is controlled inside `animateRobotTo()` and triggered via a boolean flag `catHasReacted`.

To make the reaction realistic, the cat does not always teleport randomly — it attempts to move away in the opposite direction to the robot's vector, bounded by the map area.

```
if (distanceToCat < 70.0 && !catHasReacted) {
    catHasReacted = true;
    catItem->setPixmap(QPixmap("alert.png"));
    QTimer::singleShot(500, this, [this]() {
      catItem->setPixmap(QPixmap("run.png"));
      QPointF away = calculateAwayPosition();
      catItem->setPos(away);
    });
}
```

Additional Features:

- **Interrupt Handling**: Pressing the `Return to Dock` button sets a flag `isInterrupted`, forcing the robot to stop cleaning and go dock.
- **Timers for Events**: Timers are used for smooth animations and delayed responses (charging, FSM frames, cat reaction).
- **Scene Boundaries**: Random positions (for cat and dust) are constrained within the carpet bounds using `QRandomGenerator`.

This Qt based FSM simulation demonstrates the integration of OOP FSM logic with interactive GUI programming.

V. Conclusion

This project explored the implementation of FSMs using both C++ and the Qt framework to simulate a robot vacuum cleaner. In an OOP FSM model, I used the concepts of encapsulation, polymorphism, and event-driven behavior through dynamic state transitions. I then implemented FSM as a GUI using Qt, where the robot interacted with dynamic elements like dust and cat.